

ADS Project Report

Students: Adriana Jneid, Edmond Ilbawi, Matthew Tannous, Peter Fahed

Overview:

This project is a social media platform developed in C++. This project represents a simplified version of platforms such as Instagram, Facebook, or LinkedIn. These platforms depend heavily on user interaction, fast access to data, relationship management, content sharing, and real-time engagement. The project focuses on building the core structure needed to support these functions. The platform allows users to register, log in, store posts, manage friendships, create comments, receive notifications, and maintain conversations between users. The objective of this project is to design and implement a social media platform that demonstrates how different data structures can be used to manage users, relationships, posts, comments, notifications, and messages efficiently.

The project focuses on:

- Registering and logging in users.
- Storing users efficiently using a hash map.
- Representing friendships using adjacency lists.
- Storing posts and comments using linked lists.
- Managing notifications using a stack.
- Supporting private conversations using lists.
- Creating a modular C++ program using classes and header files.

The project objectives are:

1. Allow users to register with a username and password.
2. Allow existing users to log in.
3. Store user accounts in an efficient way.
4. Support friendships between users.
5. Allow users to create and store posts.
6. Allow posts to include comments.
7. Support notifications for user interactions.
8. Support conversations between users.
9. Organize the system into separate modules for easier maintenance.
10. Provide a foundation that can be expanded into a more complete social media application.

Features and Data Structures Used:

1. User Registration, Login, and Storage

- Users can create a new account using a username and password.
- Existing users can log in using their username and password.
- The system checks whether a username already exists before allowing registration.
- The system checks whether the entered password matches the stored password during login.
- All registered users are stored in the system using their usernames.

Data structure used: Hash map

Why we used it:

A hash map was used because each username is unique and can act as a key to access the user's account. This allows the system to quickly check whether a username already exists during registration and quickly find the correct user during login. Instead of searching through all users one by one, the hash map allows efficient user lookup, making it suitable for managing user accounts in a social media platform. We also do not allow duplicate usernames, ensuring that the keys remain unique.

Alternative data structure to use: Binary search tree

Another way to store users is by using a binary search tree where the username acts as the key. A BST gives $O(\log n)$ time for insertion, search, and deletion, while a hash map gives $O(1)$ on average but can drop to $O(n)$ in the worst case if many usernames hash to the same bucket. A BST also keeps usernames in sorted order, which would be useful if we wanted to list users alphabetically or search by prefix, while a hash map does not keep any order. We used a hash map because logging in and registering are the most common operations, and a fast average search is more important than alphabetical listing in this project.

2. Friend Management

- Users can add other users as friends.
- When two users become friends, both users' friend lists are updated.
- For example, if User A adds User B, then User B is added to User A's list, and User A is added to User B's list.
- This makes the friendship mutual.
- The friendship system represents a social network, where each user is connected to other users.

Data structure used: Adjacency list

Why we used it:

An adjacency list was used because friendships in a social media platform can be represented as a graph. In this graph, each user is a vertex, and each friendship is an edge. Each user stores a set of their friends, which represents the users directly connected to them. Since friendship is mutual, the system updates both users' sets when a friendship is created. This is more efficient than using an adjacency matrix because each user is usually connected to only some users, not every user in the system.

Alternative data structure to use: Edge list

Since friendship is mutual, it can be represented by an undirected graph. Therefore, another way to manage friends is to store them in an edge list, with each edge node containing in its vertices the usernames of the two users who are friends. This approach uses less memory because the friendship will only need to be stored once, but it has a higher time complexity. Since the friends Adjacency list uses sets instead of arrays or linked lists, adding, removing and finding friends requires $O(\log n)$ time in the current implementation, while an edge list requires $O(1)$ for insertion and $O(E)$ (number of edges) for searching and removing.

3. Close Friend Management

- Users can have a separate close friends list.
- Close friends are stored separately from regular friends.
- This allows the system to represent stronger or more private relationships between users.
- This feature can later be used for privacy-based content sharing.

Data structure used: Adjacency List

Why we used it:

Close friends are represented in the same way as normal friends, so an adjacency list was also used to represent friendships. Another set was created to keep close friends separate from normal friends, which makes the user relationship system more organized.

Alternative data structure to use: Edge list

Since close friendship is also mutual, it can be represented by an undirected graph. Therefore, another way to manage close friends is to store them in an edge list similar to the way we can store normal friends.

4. Posts

- Users can create posts.

- Each post stores the username of the creator, the post description, the number of likes, and a unique post ID.
- Posts are stored under the user who created them.
- The unique post ID helps the system identify each post correctly.

Data structure used: Linked list

Why we used it:

A linked list was used because posts can be added and removed easily. This is suitable for a social media platform because users may create or delete posts at different times.

Alternative data structure to use: Dynamic array (vector)

Another way to store posts is by using a dynamic array such as a vector. A vector keeps its elements in one continuous block of memory, which makes iteration faster and allows $O(1)$ access to any post using its index. A linked list needs $O(n)$ time to reach a specific post and uses extra memory for the pointer inside each node. However, a linked list can add or remove a post at a known position in $O(1)$ time, while a vector may need to shift elements or resize, which costs $O(n)$. We used a linked list because posts are created and deleted often, and the system rarely needs to access a post by its position in the list.

5. Comments

- Users can comment on posts.
- Each post has its own list of comments.
- Each comment stores the commenter's username and the comment text.
- Comments can also have a parent comment, which allows replies to other comments.

Data structure used: Linked list

Why we used it:

A linked list was used to store comments because comments can be added continuously under a post. The parent connection gives the comment system a tree-like structure, allowing the platform to support replies and threaded discussions.

Alternative data structure to use: Forest tree

Another way to organize comments is by using a tree structure with a dummy root that represents the post. Each comment node has two pointers: a left pointer that links to the next comment at the same level (its sibling), and a right pointer that links to the first reply (its child). Top-level comments are connected through left pointers, and replies are attached to the right pointer of a comment and chained through left pointers if there are multiple replies. The advantage of this structure is that the replies to a specific comment can be accessed in time proportional to their number, while the linked list requires scanning all comments to find them. Ordering can be adjusted separately by re-

sorting the first level of sub-lists when needed, and to keep the structure consistent, comments are never deleted but only have their content replaced. The disadvantage is that this approach is more complex to implement, harder to traverse since it has no indexing, and adds more overhead than a simple linked list. We used a linked list with parent pointers because it gives the same logical structure with less code, and the current scope of the project does not need this level of detail.

6. Notifications

- Users can receive notifications when certain actions happen.
- Notifications store information such as the related username, the notification description, and whether the notification has been read.
- New notifications are placed above older notifications.

Data structure used: Stack

Why we used it:

A stack was used because notifications are usually viewed from newest to oldest. Since a stack follows the Last-In, First-Out principle, the most recent notification can be accessed first. This matches the normal behavior of notification systems in social media platforms.

Alternative data structure to use: Doubly linked list

Another way to store notifications is by using a doubly linked list, with new notifications inserted at the front. This keeps the same newest-first order as a stack but adds more flexibility. With a list, the user can scroll through older notifications without removing the newer ones, mark any notification as read no matter where it is, and delete a single notification from the middle. A stack only allows access to the top, so reaching an older notification means popping and losing all the newer ones above it. The advantage of the stack is that it is simpler and follows the Last-In, First-Out behavior automatically. We used a stack because the system only needs to display notifications from newest to oldest, but a doubly linked list would be the natural upgrade if more notification features are added later.

7. Conversations

- Users can have private conversations with each other.
- Each conversation stores the two users involved.
- The system stores the messages exchanged between them.
- Newer messages are stored first.

Data structure used: List

Why we used it:

A list was used because conversations and messages can grow over time. New messages can be

inserted efficiently, especially when they are added to the front so the newest messages appear first.

Alternative data structure to use: Hash map

Another way to store conversations is by using a hash map where the key is the other user's username and the value is the conversation object. This gives $O(1)$ lookup when a user wants to open a conversation with a specific friend, while a list needs $O(n)$ to find the right one. The advantage of a list is that it keeps conversations in the order they were added, which is useful when we want to display recent conversations first. A hash map does not keep any order, so we would need extra work to sort conversations by recency. We used a list because conversations are usually shown in order, and the number of conversations per user is normally small enough that a linear search is not a problem.

8. Messages

- Messages are stored inside conversations.
- Each message contains the message text.
- Each message also stores an indicator showing which user sent it.
- This allows the system to identify the sender without storing full sender information every time.

Data structure used: Linked List of pairs

Why we used it:

A pair was used because each message needs to store two related values: the message itself and the sender indicator. This keeps the message structure simple and efficient for two-person conversations.

Alternative data structure to use: Conversation tree (left-child / right-sibling model)

Another way to manage messages is by using a tree structure with a dummy root node that represents the messaging system. Each node has two pointers: a left pointer that links to different conversations (each one starting with the first message exchanged with a specific user), and a right pointer that links the sequence of messages within a single conversation. The left chain acts as a list of conversations with different users, while the right chain stores the actual messages of each conversation. Each message node holds the sender's username and the message text. The advantage of this structure is that it gives direct access to a specific conversation without scanning all messages, and ordering can be handled separately when needed. Deleted messages are not removed but replaced with placeholder content to keep the structure consistent. The disadvantage is that it is more complex to implement, harder to traverse since it has no indexing, and adds more overhead than a simple linear message list. We used a linked list of pairs because the current system only supports simple two-person chats where this kind of structure is not necessary.